

Homework 3

Elena Amadei & Min Zhu

March 2, 2026

Abstract

In this homework, we perform the 5 exercises provided in the pdf "hw3-2026.pdf".

Exercise 1

a

In this part, we carry out the recursive FFT to the vector $b = [1; 1; 2; 2]$, and continue the recursion until reaching Z_1 . The detailed computation process is presented as follows.

$$\hat{f} = Z_n f = \begin{bmatrix} I & \Lambda_{n/2} \\ I & -\Lambda_{n/2} \end{bmatrix} \begin{bmatrix} Z_{n/2} f^{even} \\ Z_{n/2} f^{odd} \end{bmatrix} = \begin{bmatrix} I \cdot Z_{n/2} f^{even} + \Lambda_{n/2} \cdot f^{odd} \\ I \cdot Z_{n/2} f^{even} - \Lambda_{n/2} \cdot f^{odd} \end{bmatrix}$$

Further, $Z_{n/2}$ can then be divided as

$$Z_{n/2} f^{even} = \begin{bmatrix} I & \Lambda_{n/4} \\ I & -\Lambda_{n/4} \end{bmatrix} \begin{bmatrix} Z_{n/4} f^{even,even} \\ Z_{n/4} f^{even,odd} \end{bmatrix} = \begin{bmatrix} I \cdot Z_{n/4} f^{even,even} + \Lambda_{n/4} \cdot f^{even,odd} \\ I \cdot Z_{n/4} f^{even,even} - \Lambda_{n/4} \cdot f^{even,odd} \end{bmatrix}$$
$$Z_{n/2} f^{odd} = \begin{bmatrix} I & \Lambda_{n/4} \\ I & -\Lambda_{n/4} \end{bmatrix} \begin{bmatrix} Z_{n/4} f^{odd,even} \\ Z_{n/4} f^{odd,odd} \end{bmatrix} = \begin{bmatrix} I \cdot Z_{n/4} f^{odd,even} + \Lambda_{n/4} \cdot f^{odd,odd} \\ I \cdot Z_{n/4} f^{odd,even} - \Lambda_{n/4} \cdot f^{odd,odd} \end{bmatrix}$$

Here $n = 4$, $Z_1 = w_4^0 = 1$, $\Lambda_1 = \text{diag}(1) = 1$, $\Lambda_2 = \text{diag}(1, w_4^1) = \text{diag}(1, -i)$, $f^{even} = f^{odd} = [1; 2]$, $f^{even,even} = f^{odd,even} = 1$, $f^{even,odd} = f^{odd,odd} = 2$. Thus, $Z_2 f^{even} = Z_2 f^{odd} = [3; -1]$. Substituting all the values back into the original equation, we obtain

$$\hat{f} = z_n f = \begin{bmatrix} \begin{bmatrix} 1 & \\ & 1 \end{bmatrix} \times \begin{bmatrix} 3 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 & \\ & -i \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} \\ \begin{bmatrix} 1 & \\ & 1 \end{bmatrix} \times \begin{bmatrix} 3 \\ -1 \end{bmatrix} - \begin{bmatrix} 1 & \\ & -i \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 6 \\ -1 + i \\ 0 \\ -1 - i \end{bmatrix}$$

b

Here we construct the recursive FFT function and implement it to the vector "b = tan(2*(1:16))". Initial b values and the corresponding FFT results are shown in table 1, and the detailed function can be seen in "fftx.m" file.

Table 1: Vector b and its FFT results

b	Value of b	Value of fft results
tan(2)	-2.1850	-6.4334 + 0.0000i
tan(4)	1.1578	-13.2255 - 3.7214i
tan(6)	-0.2910	4.8975 + 5.0699i
tan(8)	-6.7997	12.5234 -10.5610i
tan(10)	0.6484	-2.0524 -10.4507i
tan(12)	-0.6359	5.1091 + 9.3797i
tan(14)	7.2446	-15.1964 + 2.8270i
tan(16)	0.3006	-8.5980 +14.0977i
tan(18)	-1.1373	4.5572 + 0.0000i
tan(20)	2.2372	-8.5980 -14.0977i
tan(22)	0.0089	-15.1964 - 2.8270i
tan(24)	-2.1349	5.1091 - 9.3797i
tan(26)	1.1788	-2.0524 +10.4507i
tan(28)	-0.2814	12.5234 +10.5610i
tan(30)	-6.4053	4.8975 - 5.0699i
tan(32)	0.6610	-13.2255 + 3.7214i

c

To compare the computational cost and performance of our FFT-implementation with the naive matrix-vector multiplication of $F_N * b$, we conduct systematic CPU time comparison between the two methods. To eliminate interference from other factors and reduce uncertainty, we perform multiple(here) independent experiments and report the average running time.

With the knowledge that the complexity for naive and FFT method is $O(n^2)$ and $O(n \log n)$ respectively, we expect the FFT method to require less computational time than naive approach, thereby demonstrating its efficiency. However, when p is not large enough (for example, $p = 10, nv^{max} = 1024$), the naive method consistently outperforms the FFT implementation. As we gradually increase p , and when $p = 13, nv^{max} = 8192$, our ideal performance behavior is observed as in Fig.1.

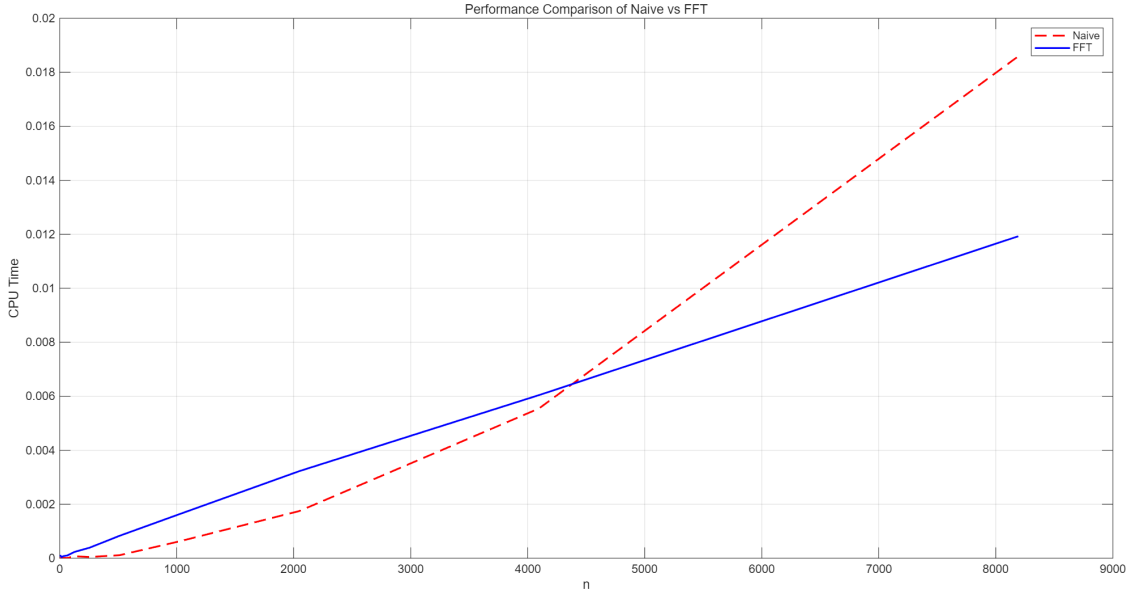


Figure 1: CPU time comparison I

We can see from this plot that, the FFT method takes more time initially. It's probably because for small problem sizes, the recursive FFT requires to decompose to the lowest level as well, introducing

additional overhead. However, as the size increases, the naive method shows a rapidly accelerating growth in runtime, while the FFT runtime grows more steadily. Around $n \approx 4500$, the times of two methods get the same and from then on, the FFT method exhibits a clear computational advantage.

d

Given the structure of vector b that $b = [y_1, y_1, y_2, y_2 \dots y_s, y_s]'$, it's obvious to find $f_{odd} = f_{even} = [f_1, f_2 \dots f_s]$. Thus in the FFT algorithm, we only need to compute once when do the first layer of division.

Theoretically, the computational cost of recursive FFT is $b_n = \frac{3}{2}n \log_2 n$, and here $n = 2s = 2^p$, leading to the standard FFT cost being

$$b_n = \frac{3}{2} \times 2^p \log_2 2^p = 3p \cdot 2^{p-1}$$

While for this special case that $f_{even} = f_{odd}$, b_n can be split to two parts: cost of $b_{n/2}$ and the reduced cost of the first layer.

$$\begin{aligned} b_{n,new} &= b_{n/2} + \frac{3}{2}n \\ &= \frac{3}{4} \times 2^p \log_2(2^{p-1}) + \frac{3}{2} \times 2^p \\ &= \frac{3}{2} \times 2^{p-1} \times (p-1) + 3 \times 2^{p-1} \\ &= \left(\frac{3}{2}p + \frac{3}{2} \right) \times 2^{p-1}. \end{aligned}$$

Hence, in theory, the computational cost of $b_{n,new}$ is slightly more than half that of b_n . We then create a new function "fftx_eveneqodd" as an improved FFT algorithm specifically optimized for cases where $f_{odd} = f_{even}$. And we add this method to the CPU-time comparison simulation as in question c) to demonstrate their performance. It is noticeable that here b is not a random vector, but one that satisfies " $f_{odd} = f_{even}$ " condition. The result can be seen in Fig.2

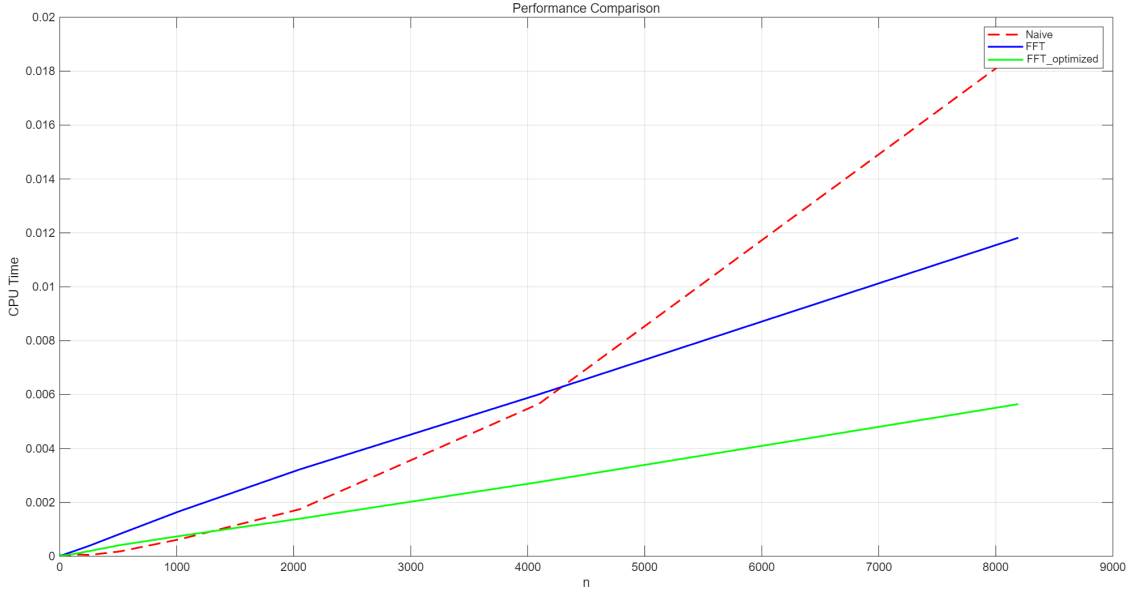


Figure 2: CPU time comparison II

It demonstrates that this optimized FFT algorithm indeed has roughly half the computational cost of the standard FFT method, as shown by the theoretical derivation before.

Exercise 2

In this task, an audio file is analyzed performing the Fast Fourier Transform (FFT) to being able to clean the sound, the Inverse FFT to make the obtained result reproducible on the time-signal scale.

a

Here, we present the semilogy plot (3) corresponding to the absolute values of the Fourier coefficients.

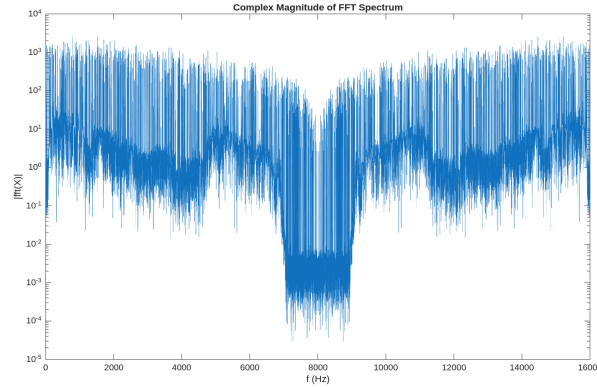


Figure 3: Absolute values of the Fourier coefficients.

b

To obtain a cleaner sound, we set to zero all the values being greater than or equal to a certain threshold. The threshold has been chosen to be the 1% of the absolute value of the maximum value of the complex Fourier coefficients, i.e. $\text{threshold} = 25.29$. The corresponding semilogy plot that was obtained can be seen below (4). By playing the sound, it can now be understood the hidden message saying "Numerics is fun".

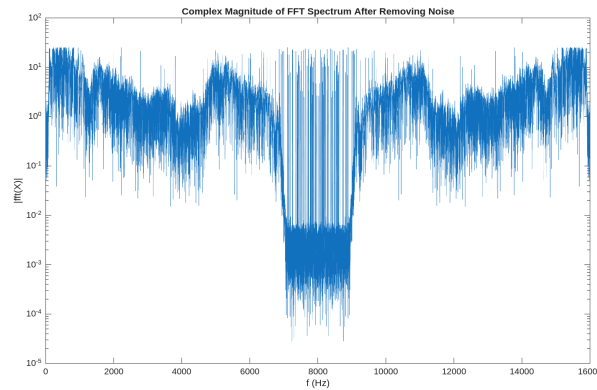


Figure 4: Absolute values of the Fourier coefficients without noise.

c

Here, it was of interest finding out what is the largest problem's size that can be solved with the naive DFT with one of our laptops under 30 minutes. We implemented the naive procedure on MATLAB, where we manually computed $\hat{f} = Z_n f$ for f being the starting vector of samples' sizes. It turned out that the largest problem size is 217,250. Hence, since the problem addressed in the task has an original signal size of 105,472, we could have indeed performed the problem within 30 minutes. Specifically, we need approximately 7 minutes to address the given problem with the Discrete Fourier Transform.

However, this would be very inconvenient, because we can perform the same procedure with a cheaper computational cost: $O(n * \log(n))$, instead of $O(n^2)$.

Exercise 3

a

Matrix A is created by the following code:

```
randn('seed',0);
v=randn(n,1);
D=diag(randn(n,1));
a = v*v';
A=v*v'+D;
```

Matrix A consists of two parts: $v*v'$ and D. Among them, $v*v'$ is an $n \times n$ matrix with rank one, since each of its columns is a scalar multiple of the vector v and hence all columns are linearly dependent. The matrix D is diagonal, containing only diagonal entries and zeros elsewhere.

According to the definition of a semi-separable matrix, we only focus on the off-diagonal submatrices, which are all elements of $v*v'$. Moreover, by the rank property that if C' is a subblock of a matrix C, then $\text{rank}(C') \leq \text{rank}(C)$.

All off-diagonal submatrices in matrix A is a submatrix of matrix $v*v'$, thus they all have rank that no more than 1. Therefore, matrix A is a semi-separable matrix of order 1.

b

In this part, we first complete the missing codes in "smatrix_skeleton.m" file, we fill them as follows and check the answer by calculating the difference of two methods.

```
% missing codes
if (~isKey(child_dict,node_key))
    b(Itau)=b(Itau)+A(Itau, Itau)*x(Itau);
...
block1_add=A(Isigma1,Isigma2)*x(Isigma2);
block2_add=A(Isigma2,Isigma1)*x(Isigma1);
```

As an optimized way of multiplication, the raw $A(I, J) * x(J)$ can be modified as

$$A(I, J) * x(J) = v(I) (v(J)' * x(J))$$

The corresponding code changes to

```
block1_add = v(Isigma1) * (v(Isigma2)' * x(Isigma2));
block2_add = v(Isigma2) * (v(Isigma1)' * x(Isigma1));
```

This version is more efficient in computation. In the original approach, the matrix-vector multiplication Ax requires $O(n^2)$ operations, since A has the dimension $n \times n$.

In the improved method we compute the inner product $v(J)' \times X(J)$ first, which yields a scalar with complexity $O(n)$. We then multiply the vector $v(J)$ by this scalar, which also costs $O(n)$. Therefore, the total computational cost of the optimized method is $O(n) + O(n) = O(n)$, which is significantly faster than the original $O(n^2)$, especially for large n.

To demonstrate this timing difference more clearly, we experiment on several values of n and get the following plot. It is obvious that, at first, the naive method has a slightly shorter computational time due to the small size of matrix A. However, as n increases, the optimized holder method demonstrates a progressively greater computational advantage and the performance gap widens with increasing matrix size.

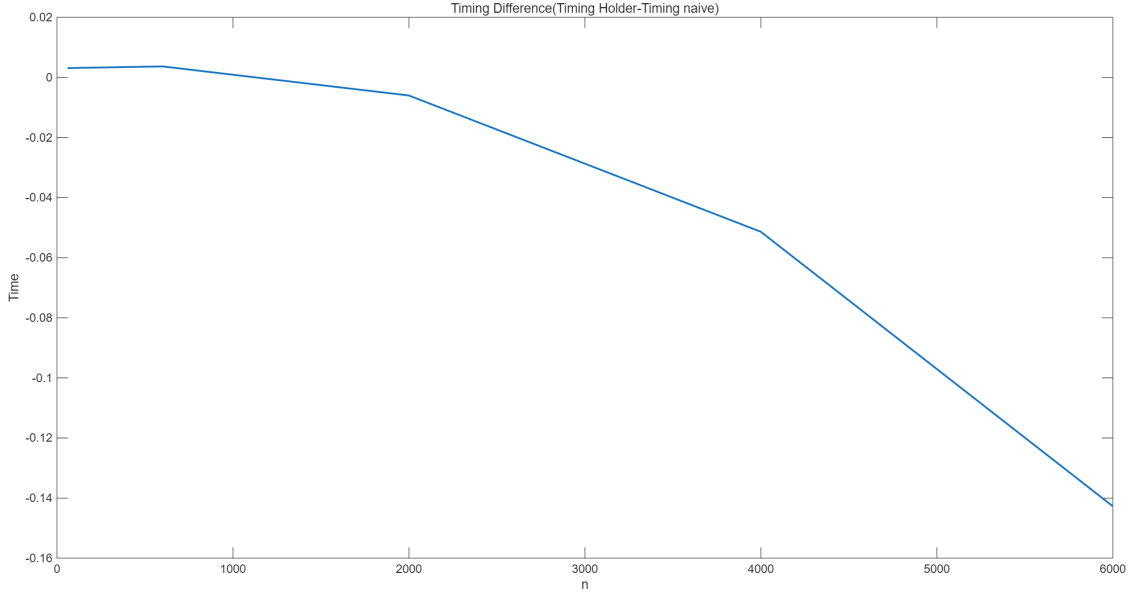


Figure 5: Timing Difference of Two Methods

c

Similar to the naive way of calculating the first three eigenvalues, we create a matrix vector product function "f_smart" based on the "matvec_hodlr" function.

```
% optimized holder
f_smat= @(x) matvec_hodlr(x, A, v, index_dict, child_dict);
[V,D] = eigs(f_smat, n, 3, 'sr');
```

We do the experiment by changing settings of matrix size n and stopsize, the initial time costs with normal n and $stopsize = 8$ are shown in Table 2.

Table 2: Eig Time Comparison I

n	Naive	HOLDR	Difference
1000	1.3223	2.6648	-1.3425
2000	4.0133	3.0026	1.0107
3000	14.2710	6.8200	7.4510
4000	30.8030	6.4596	24.3434

As the matrix size increases, the advantage of the HOLDR method becomes increasingly evident. Once HOLDR surpasses the naive approach in performance, the time gap continues to widen as n grows.

Next, we compare the computational time of the HOLDR method with MATLAB's built-in 'eigs' function. When the stopping size is set to 8, increasing the matrix dimension n leads to significantly longer running times for HOLDR, and the gap between HOLDR and the built-in method becomes larger.

Therefore, we increase the stopping size to 64 and then to 256. Repeating the experiments under these settings, we observe that the performance of the HOLDR method improves substantially. Its computational advantage gradually emerges, and for sufficiently large n , it even outperforms the built-in method.

The results corresponding to $stopsize = 256$ are reported in the table below.

Table 3: Eig Time Comparison II

n	MATLAB	HOLDR	Difference
2000	0.1297	0.4325	-0.3028
6000	1.7642	1.9415	-0.1773
9000	1,8691	1.4702	0.3989
12000	4.1157	2.5012	1.6145
15000	10.4048	5.2304	5.1745

Exercise 4

The data related to the Zalando items is once again analyzed.

a

After inspecting the given data, we were able to identify two categories of clothes: shoes and jackets. Below, you can find two examples of the categories present in the data set (??).

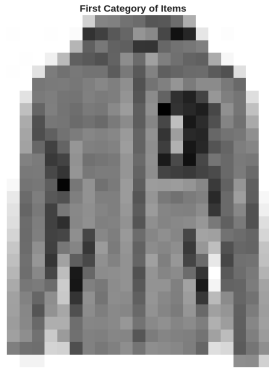


Figure 6: Image displaying first category of clothes.

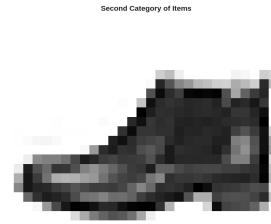


Figure 7: Image displaying second category of clothes.

b

Performing clustering lead us to find out that there are 129 incorrectly classified images. The figure 8 shows the first of these images.

c

As in (b), we performed clustering to split the items into the two categories. Now, the number of images incorrectly classified is 41. Below, we displayed the first of these images (9).

d

In the for-loop in `hw3_zalando_conv.m`, i is iterated over the total columns of the matrix *items*, i.e. 6000. Then, firstly, the data in the i -th column of items are reshaped and stored in a 28×28 matrix, X . Secondly, a zero-matrix Z (28×28) is built. Subsequently, an other for loop is created, iterating j over n , that is 28. In this inner loop, a column-vector z is computed with the function `matvec_circulant`, which takes two column-vectors as input. The function builds a circulant matrix via a Toeplitz matrix with the first inputted vector, and performs a matrix-vector multiplication with the circulant matrix and the second inputted vector. Therefore, inside the j -loop, the function takes the column-vector *filter* and the j -th column of X as input, returning z as final output vector. This loop finishes with z inserted in the j -th column of the previously created matrix Z . Finally, the i -loop ends by stacking the Z



Figure 8: First incorrectly classified image from *hw3_zalando_direct.m*.



Figure 9: First incorrectly classified image from *hw3_zalando_conv.m*

matrix into a single column-vector, and inserting it into the i -th column of the matrix object *item_conv*. Hence, these loops fill in the columns of the matrix *item_conv* with matrix-vector multiplication with circulant matrices, which corresponds to circular convolutions.

e

By performing `fft`, the function `matvec_circulant`-function performs the convolution in a faster way. Specifically the naive method took double the time to perform the matrix-vector computation compared to the `fft` approach, giving the same clustering. The code for the `fft` version of the function can be seen below.

```
function x_hat=matvec_fft(z,b) % two column vectors as input
    z_hat = fft(z);
    b_hat = fft(b);
    x_hat = ifft(z_hat .* b_hat);
end
```

f

By increasing the value of the `alpha` parameter to equal 10, the number of misclassified images drops to 27. Wherease, if it is set to 1, they increase up to 129.

Exercise 5

- **Video quiz 10a:**

This video explained how to extend a Toeplitz matrix via a circulant matrix, i.e. $T = C(z)b \Rightarrow Tb$. The version $2n \times 2n$ was presented through an example. Given a Toeplitz matrix (4×4) , a matrix C (8×8) was built s.t.

$$C = \begin{bmatrix} T & Q \\ Q & T \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} Tb \\ Qb \end{bmatrix}$$

From this extension, the first term Tb is kept, while the second term Qb is ignored.

- **Video quiz 10b:**

This video presents the problem: Given a vector b , find y , s.t. $Ty = b$, for T being a symmetric-

positiv-definite Toeplitz matrix,

$$\begin{bmatrix} r_0 & r_1 & \dots & r_{n-1} \\ r_1 & r_0 & \dots & \cdot \\ \cdot & & & \cdot \\ \cdot & & & \cdot \\ \cdot & & & r_1 \\ r_{n-1} & \dots & r_1 & r_0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \cdot \\ \cdot \\ \cdot \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \cdot \\ \cdot \\ \cdot \\ b_n \end{bmatrix}$$

Here, it can be notice that without loss of generality, we can set $r_0 = 1$, so that $\text{tr}(\lambda) = r_0 + \dots + r_0 = nr_0 > 0$, since the T matrix is symmetric-positive-definite by definition. This problem can be addressed with three different reasoning for three different algorithms: (

i) Durbins algorithm allows to solve it by

$$T_n Y = \begin{bmatrix} r_1 \\ \cdot \\ \cdot \\ r_{n-1} \\ r_n \end{bmatrix}$$

where the last term, r_n is arbitrary.

(ii) Levinson-Durbin algorithm provides the general method to solve linear systems,

$$Ty = b$$

where b is arbitrary.

(iii) Trench method algorithm is actually for the inverse of a Toeblitz matrix, T_n^{-1} .

- **Video quiz 11:**

In this video the skew-Hermitian splitting (HSS) matrices are introduced. By definition an HSS is a dense matrix, hierarchically organized, have low rank off-diagonal blocks, and nested bases. Given its cheap computational cost ($O(n)$), it allows for arithmetic and linear complexity operations, such as low rank approximations, sparse direct solvers, and gaussian processes.